🛡️ ScamShield AI — Anti-Scam Verification Platform

An AI-powered web application that detects scams, phishing links, and fraudulent messages in real time.

🚀 Features

- **Real-time Scam Detection** — Paste suspicious text, URLs, or emails and get instant results
- **OCR Support** — Upload screenshots and the app reads the text automatically
- **Google Safe Browsing API** — Checks URLs against Google's threat database
- **NLP Classification** — Naive Bayes ML model classifies content as Safe / Suspicious / High-Risk
- **Education Module** — Learn about phishing, OTP fraud, fake job scams, and more
- **Scam Reporting** — Community-driven scam report submissions

---

🧰 Tech Stack

Layer	Technology
Frontend	React 19, Vite, Tailwind CSS, Tesseract.js
Backend	Node.js, Express.js
Database	MongoDB Atlas
AI / ML	Python, Scikit-learn, Naive Bayes, TF-IDF
Security	Helmet, JWT, Rate Limiting, CORS
URL Analysis	Google Safe Browsing API, Levenshtein Distance

⚙️ Setup & Installation

1. Clone the repository
```bash
git clone https://github.com/your-username/anti_scam_verification.git
cd anti_scam_verification
```

2. Backend Setup
```bash
cd backend
npm install
```

Create a `.env` file in the `backend/` folder:
```env
PORT=5000
```

MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
SAFE_BROWSING_API_KEY=your_google_safe_browsing_api_key
```

Start the backend:
```bash
npm run dev
```

3. Frontend Setup
```bash
cd frontend
npm install
npm run dev
```

🔍 How It Works

1. User pastes text or uploads a screenshot
2. Tesseract.js extracts text from images (client-side)
3. Text is sent to the backend API
4. **Layer 1** — URLs checked via Google Safe Browsing API + Levenshtein homoglyph detection
5. **Layer 2** — Keyword scan for urgency, financial, and security indicators
6. **Layer 3** — Naive Bayes ML model gives a probability score
7. Final verdict: **Safe ✅ / Suspicious ⚠️ / High-Risk 🚨**
8. Result saved to MongoDB and returned to the user

📁 Project Structure

```
anti_scam_verification/
├── frontend/              # React app
│   └── src/App.jsx
└── backend/
    ├── server.js
    ├── ai/
    │   ├── classifier.py  # Python ML classifier
    │   └── models/        # Trained model files
    └── src/
        ├── config/        # DB connection
        ├── controllers/   # API handlers
        ├── routes/        # API endpoints
        ├── models/        # MongoDB schemas
        └── services/      # AI orchestration logic
```
```

🔒 Security

- HTTP headers secured with Helmet
- Rate limiting: 100 requests per 15 minutes per IP
- Passwords hashed with bcryptjs
- JWT-based authentication
- API keys stored in `.env` (never committed to Git)

📄 License

This project was built as part of an internship at **Ultimez
Technology**.